



VIVEKANANDA GLOBAL UNIVERSITY, JAIPUR



(Established by Act11/2012ofRajasthanGovt.Coveredu/s2(f)ofUGCAct,1956)

Lecture Notes

Course Code - UGCSE201 Course Name - Data Structures

Module 2: Stacks, Queues and Lists

Contents

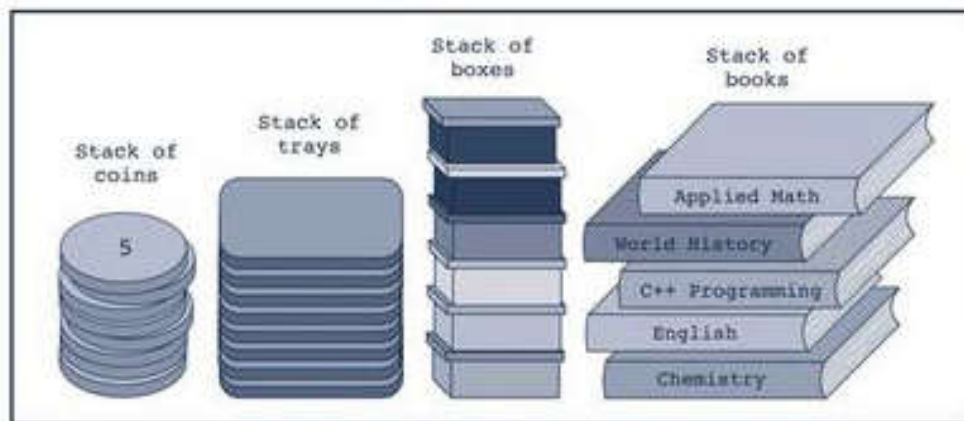
- Module 2. 1. Definition, Array based implementation of stacks
- Module 2. 2. Examples- Infix, postfix, prefix representation
- Module 2. 3. Applications- Mathematical expression Evaluation
- Module 2. 4. Queues-Array based implementation of Queues
- Module 2. 5. Circular implementation of Queues, Double ended queues.
- Module 2. 6. Linked List- Singly linked Lists and Operations associated with it
- Module 2. 7. Stack and Queue Implementation Using linked list,
- Module 2. 8. Straight circular implementation of doubly linked Lists, Priority queues

2.1.1 Introduction to Stack

In the real world, a stack refers to a physical or virtual arrangement of objects or items that follows the Last-In-First-Out (LIFO) principle. Means insertion and deletion can only take from One End i.e. TOP. It is similar to a stack of objects or papers, where you can only add or remove items from the top. Here are a few examples of stacks in real-world contexts:

- ✓ Stack of Plates: When you stack plates in a kitchen cabinet, the last plate you put on top is the first one you can easily access or remove. If you want to use a plate from the stack, you would typically take the top plate. Similarly, when you wash dishes and put them back in the cabinet, you place them on top of the stack.

- ✓ Stack of Books: If you have a stack of books on a table, the book on the top of the stack is the one you can readily pick up or read. Whenever you add a new book to the stack, it goes on top, and if you want to remove a book, you would remove it from the top of the stack.



Stack Examples

Definition

A stack is a fundamental data structure in which element may be inserted and deleted only at one end, called as top of Stack. It means that elements are removed from Stack in the reverse order of that in which they were inserted into the Stack; this manner is termed as Last-In-First-Out (LIFO) principle. It is an abstract data type that represents a collection of elements with two main operations: push and pop.

- ✓ “Push” is the term used to insert an element into a stack.
- ✓ “Pop” is the term used to delete an element from a Stack.

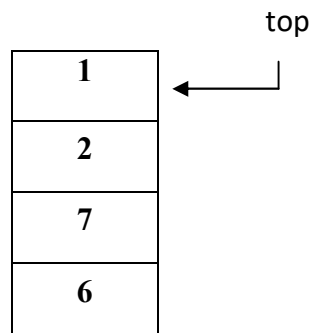
In a stack, new elements are always added or pushed onto the top, and the only element that can be removed is the most recently added element, which is popped from the top. This behaviour gives the stack its characteristic LIFO property.

Think of a stack as a physical stack of books, where you can add a new book to the top of the stack or remove the book that is currently on top. The last book you placed on the stack is the first one you can access or remove.

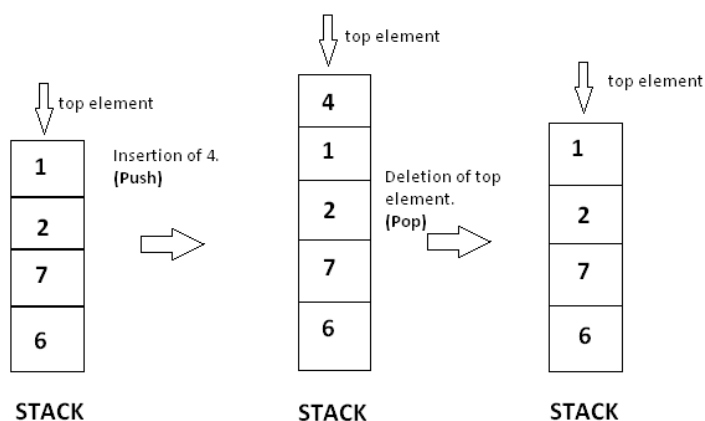
Example: Suppose 4 elements are pushed in order to, onto an empty stack:

5 , 7 , 2, 1

So the stack will look like this; top is pointing to the top element of the stack



Now see how push and pop operations executes



When push operation is performed, 4 element is inserted; and when pop operation is performed, 4 is deleted.

2.1.2 Array Implementation to Stack

In computer programming, a stack can be implemented using one-way-list or a linear array. Unless otherwise stated or implied, each of our stacks will be maintained by a linear array STACK. The array provides a simple and efficient way to store and manipulate the elements of the stack. Here's how an array can be used to represent a stack:

Define an array: Start by defining an array of a fixed size that will serve as the underlying data structure for the stack. For example, you can create an array called "STACK" with a maximum capacity of N elements.

Create a top pointer: Initialize a variable called "top" to keep track of the topmost element in the stack. Means top is basically a pointer or index which points to top element of stack. Initially, the stack is empty, so set the value of "top" to -1.

Push operation

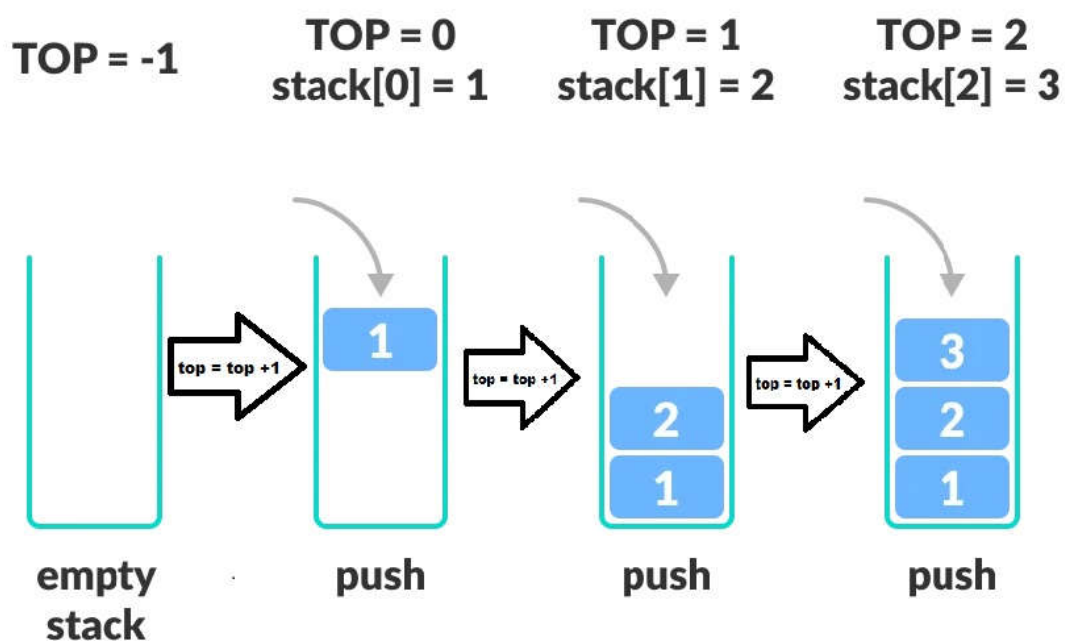
To add an element to the stack, perform the following steps:

- 1 **Overflow Condition;** Check if the stack is full by comparing the value of "top" with the maximum capacity of the array. If "top" is equal to N-1, the stack is full, and you cannot add more elements.
- 2 If the stack is not full, increment the value of "top" by 1 to move it to the next available position in the array.
- 3 Place the new element at the position specified by the value of "top" in the array.

Just look into the example;

Here an empty stack is shown; where TOP is positioned at -1 which shows stack is empty. Now when push operation is called to push "1" into the stack; first thing which needs be done is creating space for the new element; means top is incremented with 1

to point at 0th index of the Stack. So space is created with stack[0] i.e. stack[top], now place ITEM i.e. 1 on the top of the stack with stack[top]=1; this gives 1 is inserted into the top of the stack. Similarly further push operation can be seen in the diagram



Pop operation:

To remove an element from the stack, follow these steps:

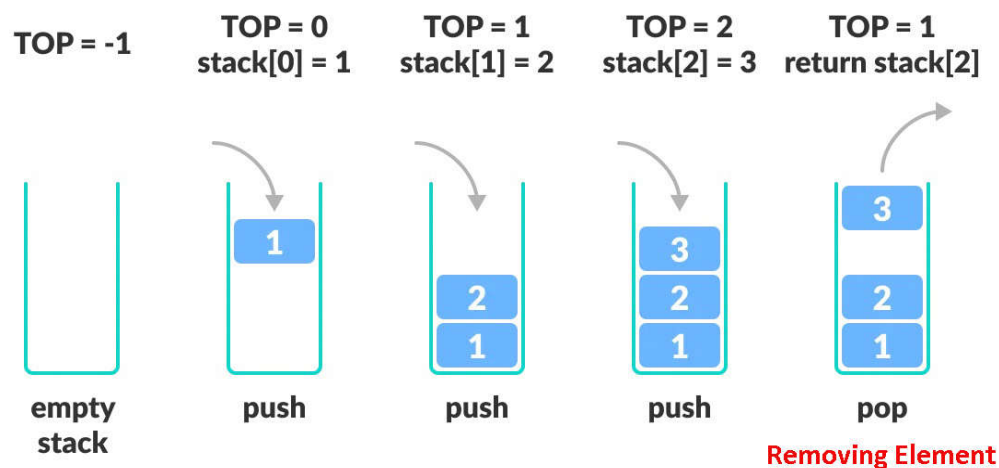
- 1 **Underflow Condition;** Check if the stack is empty by comparing the value of "top" with -1. If "top" is -1, the stack is empty, and there are no elements to remove.
- 2 If the stack is not empty, access the element at the position specified by the value of "top" in the array.
- 3 Decrement the value of "top" by 1 to move it to the previous position in the array.

Here the same stack is used, where TOP is positioned at 2 which shows on the top of the stack i.e. stack[2] is 3. Now when pop operation is called first thing which needs

be done is creating variable ITEM to store top element as before deleting we might save it for the other purpose.

ITEM = stack [top]

Now we are ready to remove the top element from stack ; simply decrement top with 1 i.e. $TOP = TOP - 1$; now TOP will point to 1 i.e. STACK[1] which is 2. Similarly push and pop operation can be seen in the diagram



Peek operation

To retrieve the topmost element without removing it, you can simply access the element at the position specified by the value of "top" in the array.

Algorithm for PUSH

PUSH (STACK, TOP, MAX, ITEM)

This procedure pushes an ITEM onto the STACK. Here MAX is the upper bound of the declared stack as array. If stack is having N element and starting index is 0, then

Max will be $N-1$.

Step 1 [Overflow Condition ?] if $TOP = MAX$, then

Print: Overflow and Return.

Step 2 Set $TOP := TOP + 1$ [Increase top with 1]

Step 3 Set $STACK[top] := ITEM$ [Insert ITEM in new TOP position]

Step 4 Return.

Algorithm for POP

POP (STACK, TOP, ITEM)

This procedure pop or remove an ITEM from the STACK. And assign the removed to ITEM.

Step 1 [Underflow Condition ? means no item to delete] if $TOP = -1$, then

Print: underflow and Return.

Step 2 Set $ITEM := STACK[TOP]$ [Assign TOP element to the ITEM]

Step 3 Set $TOP := TOP - 1$ [Decrease TOP by 1]

Step 4 Return.

Minimizing Overflow

There is an essential difference between underflow and overflow in dealing with stacks. Underflow depends exclusively upon the given algorithm and the given input data, and hence there is no direct control by the programmer.



VIVEKANANDA GLOBAL UNIVERSITY, JAIPUR



(Established by Act11/2012ofRajasthanGovt.Coveredu/s2(f)ofUGCAct,1956)

Overflow, on the other hand, depends upon the arbitrary choice of the programmer for the amount of memory space reserved for each stack, and this choice does influence the number of times overflow may occur.

Generally speaking, the number of elements in a stack fluctuates as elements are added and removed from a stack. Accordingly, the particular choice of the amount of memory for a given stack involves a time-space trade-off. Specifically, initially reserving a great deal of space for each stack will decrease the number of times overflow may occur; however, this may be an expensive use of the space if most of the space is seldom used. On the other hand, reserving a small amount of space for each stack may increase the number of times overflow occurs; and the time required for resolving an overflow, such as by adding space to the stack, may be more expensive than the space saved.

Various techniques have been developed which modify the array representation of stacks so that the amount of space reserved for more than one stack may be more efficiently used. Most of these techniques lie beyond the scope of this e-text book.